Claude Code Deep Dive Whitepaper

From Beginner to Expert: Architecture Analysis, Internals & Power User Guide

Version: Based on source code analysis of Claude Code v2.1.88
Date: April 2026
Source: In-depth reverse engineering analysis based on the public repositories `sanbuphy/claude-code-source-code` and `ChinaSiro/claude-code-sourcemap`

Part 1: Overview & Background

1.1 What Is Claude Code

Claude Code is Anthropic's official AI-powered CLI (Command Line Interface) tool for programming. It is far more than a simple chat interface — it is a **full-fledged agent framework**, featuring:

- **Full file system access** — read, edit, and create files
- **Shell command execution** — run arbitrary Bash/PowerShell commands
- **Multi-agent coordination** — the main agent can spawn sub-agents to work in parallel
- **MCP protocol integration** — extend tool capabilities through a standard protocol
- **Intelligent context management** — automatic compaction and token budget tracking
- **Permission security system** — multi-layered permission model to prevent accidental operations
- **Hook event system** — users can inject custom logic at key execution points
- **Session persistence** — resumable, interruptible long-running sessions

1.2 Technical Scale

Metric	Value
TypeScript source files	**1,884**
Total lines of code	**~513,000 LOC**
Tool implementations	**45+**
Slash commands	**80+**
Service modules	**38**
MCP transport protocols supported	**6**
Hook event types	**15+**
Feature gate modules	**108**

1.3 Technology Stack

```
Language:       TypeScript (ES2022 target)
Runtime:        Node.js >= 18 / Bun (internal)
UI Framework:   React + Ink (terminal UI rendering)
```

```
Build Tool:     esbuild + custom transform pipeline
Package Mgmt:   npm (ESM modules)
API:            Anthropic Messages API
Protocol:       MCP (Model Context Protocol)
```

Part 2: Project Structure Overview

2.1 Top-Level Directory Structure

```
claude-code/
├── src/                          # Main application source (1,884 file
│   ├── entrypoints/              # Entry points
│   │   └── cli.tsx               # CLI main entry (~300 lines)
│   ├── main.tsx                  # Main orchestrator (~4,683 lines)
│   ├── query.ts                  # Core query loop (~1,729 lines) ★ Mo
│   ├── QueryEngine.ts            # Query engine wrapper
│   ├── Tool.ts                   # Tool base interface
│   ├── tools.ts                  # Tool registry
│   ├── commands.ts               # Command registry
│   ├── context.ts                # Context assembly
│   │
│   ├── tools/                    # Tool implementations (~45)
│   │   ├── AgentTool/            # Agent tool (17 files, 400KB)
│   │   ├── BashTool/             # Bash execution
│   │   ├── FileEditTool/         # File editing (8 files)
│   │   ├── FileReadTool/         # File reading
│   │   ├── FileWriteTool/        # File writing
│   │   ├── GlobTool/             # File search
│   │   ├── GrepTool/             # Content search
│   │   ├── WebFetchTool/         # Web fetching
│   │   ├── WebSearchTool/        # Web search
│   │   ├── MCPTool/              # MCP tool bridge
│   │   ├── NotebookEditTool/     # Jupyter editing
│   │   ├── SkillTool/            # Skill invocation
│   │   └── ...                   # More tools
│   │
│   ├── commands/                 # Slash commands (~80)
│   │   ├── commit.ts             # /commit
│   │   ├── review.ts             # /review
│   │   ├── mcp/                  # /mcp management
│   │   ├── config/               # /config configuration
│   │   ├── tasks/                # /tasks management
│   │   └── ...
│   │
│   ├── services/                 # Business services (38 directories)
│   │   ├── api/                  # API communication layer
│   │   │   ├── claude.ts         # Anthropic API client (125KB)
│   │   │   ├── withRetry.ts      # Retry logic (28KB)
│   │   │   └── errors.ts         # Error handling
│   │   ├── mcp/                  # MCP service (25 files, 900KB)
│   │   │   ├── client.ts         # MCP client (119KB)
```

```
│   │   │   ├── config.ts          # MCP configuration
│   │   │   ├── auth.ts            # OAuth authentication (88KB)
│   │   │   └── types.ts           # Type definitions
│   │   ├── analytics/             # Telemetry & analytics
│   │   ├── compact/               # Context compaction
│   │   ├── tools/                 # Tool execution service
│   │   │   └── StreamingToolExecutor.ts  # Streaming tool executor
│   │   └── ...
│   │
│   ├── utils/                     # Utility functions (30+ groups)
│   │   ├── permissions/           # Permission system (26 files, 720KB)
│   │   ├── hooks/                 # Hook execution
│   │   ├── settings/              # Configuration management (50+ files
│   │   ├── config.ts              # Configuration hierarchy
│   │   ├── claudemd.ts            # CLAUDE.md parsing
│   │   ├── tokens.ts              # Token counting
│   │   └── ...
│   │
│   ├── components/                # React/Ink terminal UI components (4
│   ├── state/                     # Application state management
│   ├── types/                     # Type definitions
│   ├── hooks/                     # React Hooks
│   ├── constants/                 # Constants & prompt templates
│   │   ├── prompts.ts             # System prompt construction
│   │   └── systemPromptSections.ts  # Prompt caching
│   ├── skills/                    # Skill system
│   ├── tasks/                     # Task management
│   │
│   ├── coordinator/               # Multi-agent coordination (Feature-g
│   ├── assistant/                 # KAIROS autonomous mode
│   ├── bridge/                    # Remote control
│   ├── daemon/                    # Background daemon
│   ├── remote/                    # Remote triggers
│   ├── voice/                     # Voice features
│   └── vim/                       # Vim integration
│
├── scripts/                       # Build scripts
│   ├── build.mjs                  # Main build pipeline
│   ├── prepare-src.mjs            # Source preprocessing
│   └── transform.mjs              # Code transformations
├── vendor/                        # Native module stubs
├── stubs/                         # Bun compile-time internal feature s
├── tools/                         # External tools (ripgrep, etc.)
├── package.json                   # v2.1.88
└── tsconfig.json                  # ES2022 configuration
```

2.2 Key Files Ranked by Size

Rank	File	Size	Purpose
1	`main.tsx`	803 KB	Main orchestrator, REPL startup

2	`tools/AgentTool/AgentTool.tsx`	233 KB	Agent tool core
3	`services/api/claude.ts`	125 KB	API communication client
4	`services/mcp/client.ts`	119 KB	MCP client
5	`services/mcp/auth.ts`	88 KB	MCP OAuth authentication
6	`query.ts`	68 KB	Core query loop
7	`utils/permissions/permissions.ts`	52 KB	Permission engine
8	`utils/permissions/yoloClassifier.ts`	52 KB	Auto mode classifier
9	`utils/analyzeContext.ts`	42 KB	Context analysis
10	`services/mcp/config.ts`	51 KB	MCP configuration management

Part 3: Core Architecture Deep Dive

3.1 Layered System Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                          CLI Entry Layer                             │
│                  (cli.tsx → main.tsx → REPL)                         │
├──────────────────────────────────────────────────────────────────────┤
│                         Command Dispatch                             │
│              (/commands → processUserInput)                          │
├──────────────────────────────────────────────────────────────────────┤
│                          Query Engine                                │
│                (QueryEngine.ts → query.ts)                           │
│      ┌──────────────────────────────────────────────┐                │
│      │      Core Agent Loop (async generator)       │                │
│      │   ┌──────────┐   ┌──────────┐   ┌──────────┐ │               │
│      │   │API Call  │→ │Tool Exec │→ │Context   │ │               │
│      │   │claude.ts │   │Streaming │   │Mgmt      │ │               │
│      │   │          │   │ToolExec  │   │compact   │ │               │
│      │   └──────────┘   └──────────┘   └──────────┘ │               │
│      └──────────────────────────────────────────────┘                │
├──────────────────────────────────────────────────────────────────────┤
│                           Tool Layer                                 │
│    ┌────────┐  ┌────────┐  ┌────────┐  ┌──────────┐                │
│    │  Bash  │  │  File  │  │  Web   │  │   MCP    │                │
│    │  Tool  │  │  Tools │  │  Tools │  │  Bridge  │                │
│    └────────┘  └────────┘  └────────┘  └──────────┘                │
├──────────────────────────────────────────────────────────────────────┤
│                        Permission Layer                              │
│      ┌──────────────┐  ┌──────────┐  ┌──────────────┐               │
│      │   Rules      │  │Classifier│  │   Hook       │               │
│      │   Engine     │  │          │  │  Decision    │               │
│      └──────────────┘  └──────────┘  └──────────────┘               │
├──────────────────────────────────────────────────────────────────────┤
```

```
│                    State & Config                          │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐   │
│   │ AppState │  │ Settings │  │ CLAUDE.md│  │ Session  │   │
│   │  Store   │  │ Hierarchy│  │  Memory  │  │ Storage  │   │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘   │
├──────────────────────────────────────────────────────────────┤
│                   External Services                        │
│    ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐  │
│    │ Anthropic API│  │ MCP Server   │  │ GrowthBook/      │  │
│    │ (Messages)   │  │ (Stdio/SSE)  │  │ Feature Flags    │  │
│    └──────────────┘  └──────────────┘  └──────────────────┘  │
└──────────────────────────────────────────────────────────────┘
```

3.2 Startup Flow in Detail

Phase 1: Fast-Path Detection (cli.tsx)

```
┌─ CLI Entry ──────────────────────────────────────────────────┐
│                                                              │
│  1. profileCheckpoint('main_tsx_entry')                      │
│  2. startMdmRawRead()        // Start MDM read in parallel  │
│  3. startKeychainPrefetch()  // Start keychain read in      │
│                              // parallel                     │
│                                                              │
│  Fast paths (zero module loading):                           │
│  ├─ --version        → Print version and exit immediately    │
│  ├─ --dump-system-prompt → Output system prompt              │
│  ├─ --chrome-native-host → Chrome extension mode             │
│  ├─ --computer-use-mcp → Computer Use MCP server             │
│  ├─ --daemon-worker  → Daemon worker thread                  │
│  ├─ remote-control   → Remote control bridge mode            │
│  ├─ daemon           → Daemon mode                           │
│  └─ ps / logs        → Background session management         │
│                                                              │
│  ↓ No match → Load full initialization                       │
└──────────────────────────────────────────────────────────────┘
```

Phase 2: Full Initialization (main.tsx)

```
┌─ Main Initialization ──────────────────────────────────────┐
│                                                              │
│  Lines 1-200:    Imports & performance checkpoints           │
│  Lines 200-500:  OAuth config & security settings loading    │
│  Lines 500-1000: GrowthBook feature flag initialization      │
│  Lines 1000-2000: Commander.js CLI argument parsing          │
│  Lines 2000-3700: Mode selection (interactive/headless/      │
│                   Print)                                     │
│  Lines 3700+:    REPL startup & interactive loop             │
│                                                              │
│  Parallel prefetching (after first render):                  │
│  ├─ initUser()                  // User profile              │
│  ├─ getUserContext()            // CLAUDE.md files           │
```

```
│  ├─ getRelevantTips()           // Feature tips            │
│  ├─ countFilesRoundedRg()       // Project file count      │
│  ├─ refreshModelCapabilities()  // Model capability matrix │
│  ├─ settingsChangeDetector.initialize() // Config watcher  │
│  └─ skillChangeDetector.initialize()    // Skill hot-      │
│                                         // reload          │
│                                                            │
└────────────────────────────────────────────────────────────┘
```

3.3 Core Query Loop (query.ts — The Heart of the System)

This is the single most critical file in all of Claude Code. The 1,729-line `query.ts` implements the **complete agent loop**:

```
// Core state definition
type State = {
  messages: Message[]                      // Conversation history
  toolUseContext: ToolUseContext           // Tool execution context
  autoCompactTracking?: AutoCompactTrackingState
  maxOutputTokensRecoveryCount: number     // API recovery loop counter
  hasAttemptedReactiveCompact: boolean     // Whether reactive compactio
  pendingToolUseSummary?: Promise<...>     // Pending tool use summary
  stopHookActive?: boolean                // Whether the stop hook is ac
  turnCount: number                       // Current turn number
  transition?: Continue                    // Recovery path
}

// Main generator function
async function* query(params: QueryParams)
  // yields: StreamEvent | RequestStartEvent | Message | TombstoneMessa
  // returns: Terminal { ... }

async function* queryLoop(params, consumedCommandUuids)
  // Implements: Core agent loop
  // Handles: Tool execution, compaction, recovery
```

Single-Turn Execution Flow

```
User Input
    │
    ▼
┌──────────────────────────────────────────┐
│ 1. Assemble system prompt + context      │
│    ├─ Base system instructions (role,    │
│    │   capabilities)                     │
│    ├─ Tool definitions + MCP tool hints  │
│    ├─ Permission rules (auto mode)       │
│    ├─ Memory files (CLAUDE.md)           │
│    └─ User/system context                │
├──────────────────────────────────────────┤
│ 2. Normalize messages → API format       │
├──────────────────────────────────────────┤
│ 3. Call claude.ts streaming API          │
```

```
│     ├─ text_delta events                    │
│     ├─ content_block_start/delta/stop       │
│     └─ Collect into complete message        │
├──────────────────────────────────────────────┤
│ 4. Check stop_reason                         │
│     ├─ "end_turn" → Exit loop               │
│     ├─ "tool_use" → Enter tool execution    │
│     └─ "max_tokens" → Recovery mechanism    │
├──────────────────────────────────────────────┤
│ 5. Tool Execution (StreamingToolExecutor)   │
│     ├─ Extract tool_use blocks               │
│     ├─ Permission check (canUseTool)         │
│     ├─ Execute tools in parallel             │
│     ├─ Stream progress updates               │
│     └─ Collect tool_result blocks            │
├──────────────────────────────────────────────┤
│ 6. Context Management                        │
│     ├─ Check token budget                    │
│     ├─ Trigger auto-compaction?              │
│     └─ Update message array                  │
├──────────────────────────────────────────────┤
│ 7. Continuation decision (7 branches)        │
│     ├─ Normal tool loop continues            │
│     ├─ Token budget exceeded → auto-         │
│     │   continue                             │
│     ├─ max_output_tokens → upgrade retry     │
│     ├─ Reactive compaction triggered          │
│     ├─ Stop hook needs execution             │
│     ├─ Context collapse triggered             │
│     └─ Task budget exhausted                 │
└──────────────────────────────────────────────┘
    │
    ▼
  Back to step 1 (or end)
```

3.4 Three Execution Modes

Mode	Trigger	Characteristics
Interactive REPL	Default startup	Full TUI, persistent session, interruptible
Headless/Print mode	`-p` / `--print` flag	Single query, output to stdout
SDK/Non-interactive mode	`--init-only`	Minimal initialization, no user prompts

---

Part 4: Tool System

4.1 Tool Interface Definition (Tool.ts)

```
// Tool definition type
type Tool = {
  name: string                                  // Tool name
```

```
  description: string                        // Description for the mode
  input_schema: ToolInputJSONSchema          // JSON Schema input spec
  isEnabled?: () => boolean                  // Whether enabled
  isIncludedInSystemPrompt?: () => boolean   // Whether in system prompt
  validateInput?: (input) => ValidationResult  // Input validation
  expandedDescription?: string               // Extended description
  visibility?: 'public' | 'internal'         // Visibility
  mcpInfo?: { serverName; toolName }         // MCP information
}

// Tool execution context
type ToolUseContext = {
  options: {
    commands: Command[]
    tools: Tools
    mcpClients: MCPServerConnection[]
    agentDefinitions: AgentDefinitionsResult
    thinkingConfig: ThinkingConfig
    maxBudgetUsd?: number
    // ... 20+ fields
  }
  abortController: AbortController
  readFileState: FileStateCache
  getAppState(): AppState
  setAppState(f: (prev) => AppState): void
}

// Tool result type
type ToolResult<Output> = {
  content: ContentBlockParam[]    // API-format content
  successMessage?: string         // Success message
  toolUseID?: string             // Tool use ID
  contentReplaced?: boolean       // Whether content was replaced
  suppressOutput?: boolean        // Whether to hide output
  // ... 15+ more fields
}
```

4.2 Complete Tool Inventory

File Operations (6)

Tool	Function	Key Features
FileRead	Read files	Supports images/PDF/Jupyter, line number ranges
FileWrite	Create/overwrite files	Must read before overwriting
FileEdit	Precise string replacement	`old_string` → `new_string`, supports `replace_all`
Glob	File pattern matching	Supports glob patterns like `**/*.ts`
Grep	Content search	Based on ripgrep, supports regex, context lines
NotebookEdit	Jupyter editing	Edit `.ipynb` cells

Execution (5)

Tool	Function	Key Features
Bash	Shell command execution	Sandbox support, timeout control, background execution
PowerShell	Windows commands	Windows platform only
REPL	Interactive REPL	Internal only (ANT-only)
Skill	Skill invocation	Execute predefined/user-defined skills
Agent	Agent spawning	Create sub-agents for parallel work

Networking (3)

Tool	Function	Key Features
WebFetch	Fetch web content	URL access, content extraction
WebSearch	Web search	Search engine integration
WebBrowser	Browser automation	Feature-gated, automated operations

MCP Bridge (4)

Tool	Function	Key Features
MCPTool	MCP tool proxy	Bridges MCP server tools
ListMcpResources	List MCP resources	Discover available resources
ReadMcpResource	Read MCP resources	Fetch resource contents
ListPeers	List UDS peers	Internal communication

Task Management (6)

Tool	Function
TaskCreate	Create a task
TaskGet	Get task details
TaskUpdate	Update task status
TaskList	List all tasks
TaskStop	Stop a task
TaskOutput	Get task output

System (5)

Tool	Function
AskUserQuestion	Ask the user a question
ToolSearch	Search for lazily loaded tools
EnterPlanMode	Enter planning mode
ExitPlanMode	Exit planning mode
ConfigTool	Configuration management

4.3 Tool Registration Mechanism (tools.ts)

```
// Dynamically build the available tool list
export function getAllBaseTools(): Tools {
  return [
    AgentTool,
    TaskOutputTool,
    BashTool,
    ...(hasEmbeddedSearchTools() ? [] : [GlobTool, GrepTool]),
    ExitPlanModeV2Tool,
    FileReadTool,
    FileEditTool,
    FileWriteTool,
    NotebookEditTool,
    WebFetchTool,
    TodoWriteTool,
    WebSearchTool,
    TaskStopTool,
    AskUserQuestionTool,
    SkillTool,
    // Feature-gated conditional tools
    ...(SleepTool ? [SleepTool] : []),
    ...(cronTools ? cronTools : []),
    ...(RemoteTriggerTool ? [RemoteTriggerTool] : []),
    // ...
  ]
}

// Filter tools by deny rules
export function filterToolsByDenyRules<T extends { name: string }>(
  tools: readonly T[],
  permissionContext: ToolPermissionContext
): T[] {
  return tools.filter(tool => !getDenyRuleForTool(permissionContext, to
}
```

4.4 Tool Execution Pipeline (StreamingToolExecutor)

```
Claude API returns tool_use blocks
         |
         ▼
┌──────────────────────────┐
| Extract all tool_use     |
| blocks (may be multiple  |
| in parallel)             |
└──────────┬───────────────┘
           |
    ┌──────┴──────┐
    ▼             ▼
┌──────────┐  ┌──────────┐
| Perm.    |  | Perm.    |   (parallel)
```

```
│ Check    │   │ Check    │
│ Tool A   │   │ Tool B   │
└──────────┘   └──────────┘
     │              │
     ▼              ▼

┌──────────┐   ┌──────────┐
│ Execute  │   │ Execute  │    (parallel, with
│ Tool.A   │   │ Tool.B   │     concurrency control)
│ .call()  │   │ .call()  │
└──────────┘   └──────────┘
     │              │
     ▼              ▼

┌──────────────────────────┐
│ Collect tool_result      │
│ blocks, append to        │
│ message array            │
└──────────────────────────┘
```

Part 5: Permissions & Security Model

5.1 Permission Modes

Claude Code provides **5 external permission modes** and **2 internal permission modes**:

```
External modes (user-selectable):
├── default          → Interactive prompt (default)
├── acceptEdits      → Auto-approve file edits
├── bypassPermissions → Dangerous: approve all without prompting
├── dontAsk          → Silently deny
└── plan             → Planning mode (/plan command only)

Internal modes (system use):
├── auto             → Classifier-driven decisions (Feature-gated)
└── bubble           → Internal state
```

5.2 Permission Decision Flow

```
Tool call request
     │
     ▼

┌──────────────────────┐
│ 1. Check Allow rules│ ← alwaysAllowRules in settings.json
│    Match? → Allow    │
└──────────────────────┘
           │ No match
           ▼

┌──────────────────────┐
│ 2. Check Deny rules  │ ← alwaysDenyRules in settings.json
│    Match? → Deny     │
└──────────────────────┘
           │
```

```
          | No match
          ▼
┌──────────────────────┐
| 3. Check Ask rules   | ← alwaysAskRules in settings.json
|    Match? → Prompt   |
└──────────────────────┘
          | No match
          ▼
┌──────────────────────┐
| 4. Check perm. mode |
|    bypass → Allow    |
|    dontAsk → Deny    |
|    auto → Classifier|
|    default → Prompt  |
└──────────────────────┘
          | auto mode
          ▼
┌──────────────────────┐
| 5. YOLO Classifier   |
|    safe → Allow      |
|    unsafe → Prompt   |
|    unknown → Prompt  |
└──────────────────────┘
```

5.3 Permission Rule Format

```
{
  "permissions": {
    "rules": {
      "allow": [
        "Bash(prefix:npm test)",
        "Bash(prefix:git status)",
        "FileEdit",
        "FileRead",
        "Glob",
        "Grep"
      ],
      "deny": [
        "Bash(prefix:rm -rf /)",
        "Bash(prefix:sudo)"
      ]
    }
  }
}
```

Rule matching syntax:

- `ToolName` — matches the entire tool
- `ToolName(prefix:xxx)` — matches tool name + argument prefix
- `MCP:ServerName` — matches an MCP server

5.4 YOLO Classifier (Core of Auto Mode)

```
// yoloClassifier.ts — analyzes command safety
classifyYoloAction(command: string, cwd: string)
  → 'safe' | 'unsafe' | 'unknown'

// Dangerous pattern detection:
// ├─ File system operations outside the working directory
// ├─ Credential/environment variable exposure
// ├─ System administration commands
// ├─ Network operations
// ├─ rm -rf, git reset --hard, git push --force
// ├─ drop, delete, truncate
// └─ sudo, --no-verify
```

5.5 Sandbox Support

```
// sandbox-adapter.ts — Bash sandboxing
SandboxManager.isSandboxingEnabled()
SandboxManager.areUnsandboxedCommandsAllowed()
SandboxManager.isAutoAllowBashIfSandboxedEnabled()

// Capability restrictions:
// ├─ File access whitelist/blacklist
// ├─ Network restrictions
// └─ Environment variable filtering
```

Part 6: Hook Event System

6.1 Hook Event Types

Claude Code provides **15+ hook event types**, allowing users to inject custom logic at key execution points:

Event	Trigger Timing	Typical Use Case
`SessionStart`	Session begins	Initialize environment, load config
`UserPromptSubmit`	Before user input is submitted	Input validation, logging
`PreToolUse`	Before tool execution	Permission control, parameter modification
`PostToolUse`	After tool execution	Result processing, logging
`PostToolUseFailure`	After tool execution failure	Error handling, alerts
`PermissionRequest`	Before permission dialog	Automatic decision-making
`PermissionDenied`	User denies permission	Record denial reason
`Notification`	Notification sent	Custom notification channels
`Setup`	First-run setup	Environment preparation
`Elicitation`	URL fetch request	MCP URL handling
`CwdChanged`	Working directory changed	Auto-reload configuration

`FileChanged`	Watched file changed	Hot reload
`SubagentStart`	Sub-agent starts	Track/limit sub-agents
`WorktreeCreate`	Worktree created	Isolated environment management

6.2 Hook Configuration

Three types of hooks can be configured in `settings.json`:

```
{
  "hooks": [
    {
      "event": "PreToolUse",
      "matcher": "Bash",

      "handler": {
        "type": "script",
        "command": "~/.claude/hooks/check-bash.sh"
      }
    },
    {
      "event": "PostToolUse",

      "handler": {
        "type": "javascript",
        "code": "module.exports = async (input) => { /* ... */ }"
      }
    },
    {
      "event": "UserPromptSubmit",

      "handler": {
        "type": "agent",
        "prompt": "Validate whether the user input is safe...",
        "timeout": 30,
        "model": "claude-sonnet-4-6"
      }
    }
  ]
}
```

6.3 Hook Response Protocol

```
// Synchronous hook response
{
  continue?: boolean,           // Whether to continue execution
  suppressOutput?: boolean,     // Hide output
  stopReason?: string,          // Stop reason
  decision?: 'approve' | 'block',  // Approval decision
  reason?: string,              // Decision reason
  systemMessage?: string,       // Warning message
```

```
  hookSpecificOutput?: {
    hookEventName: string,
    permissionDecision?: 'allow' | 'deny' | 'ask',
    updatedInput?: Record<string, unknown>,  // Modified input
    updatedMCPToolOutput?: unknown,
    watchPaths?: string[],
    additionalContext?: string
  }
}

// Asynchronous hook response
{
  async: true,
  asyncTimeout?: number    // Timeout in seconds
}
```

6.4 Hook Practical Scenarios

Scenario 1: Auto-approve safe commands

```
{
  "event": "PreToolUse",
  "matcher": "Bash",
  "handler": {
    "type": "script",
    "command": "echo '{\"hookSpecificOutput\":{\"permissionDecision\":\
  }
}
```

Scenario 2: Log all tool calls

```
{
  "event": "PostToolUse",
  "handler": {
    "type": "javascript",
    "code": "module.exports = async ({tool, input, output}) => { requir
  }
}
```

Scenario 3: AI-powered code change review

```
{
  "event": "PostToolUse",
  "matcher": "FileEdit",
  "handler": {
    "type": "agent",
    "prompt": "Review this file edit for security vulnerabilities. Chec
    "timeout": 60
  }
}
```

Part 7: MCP Protocol Integration

7.1 What Is MCP

MCP (Model Context Protocol) is an open standard protocol that enables AI applications to interact with external data sources and tools. Claude Code includes a complete built-in MCP client implementation.

7.2 Supported Transport Protocols

Protocol	Type	Use Case
Stdio	Process communication	Local tool servers
SSE	HTTP Server-Sent Events	Remote servers
HTTP	Request/Response	REST API integration
WebSocket	Bidirectional communication	Real-time interactive services
SDK	In-process	Internal integration
ClaudeAI Proxy	Proxy	Claude.ai cloud

7.3 MCP Server Configuration

```
// ~/.claude/mcp.json or mcpServers in settings.json
{
  "mcpServers": {
    "filesystem": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/path/
    },
    "github": {
      "type": "sse",
      "url": "https://mcp.github.com/sse",
      "headers": {
        "Authorization": "Bearer ${GITHUB_TOKEN}"
      }
    },
    "database": {
      "type": "http",
      "url": "https://db-mcp.example.com/mcp",
      "oauth": {
        "clientId": "xxx",
        "authServerMetadataUrl": "https://auth.example.com/.well-known/
      }
    }
  }
}
```

7.4 MCP Tool Integration Flow

```
At startup
  |
  ▼

┌──────────────────────────────────┐
| 1. Discovery: List all MCP tools|
|    client.tools/list             |
├──────────────────────────────────┤
| 2. Integration: Wrap as Claude   |
|    tools                         |
|    MCPTool ← { serverName,       |
|               toolName,          |
|               inputSchema }      |
├──────────────────────────────────┤
| 3. Registration: Add to main     |
|    tool registry                 |
|    getAllBaseTools() + mcpTools  |
├──────────────────────────────────┤
| 4. Execution: Route to MCP when  |
|    model invokes tool            |
|    MCPTool.call(input)           |
|    → client.tools/call           |
|    → Deserialize result          |
|    → Return ToolResult           |
├──────────────────────────────────┤
| 5. Streaming: Support server-    |
|    push responses                |
|    SSE/WebSocket continuous      |
|    receive                       |
└──────────────────────────────────┘
```

7.5 MCP Resource Prefetching

```
// Asynchronously prefetch all MCP resources
prefetchAllMcpResources(mcpClients: MCPServerConnection[]): Promise<voi
  // Parallelize resource/list calls
  // Cache results for subsequent access
  // Gracefully handle timeouts
```

Part 8: Configuration System

8.1 Four-Level Configuration Hierarchy

```
Priority from highest to lowest:

┌──────────────────────────────────────────┐
| 1. Managed Settings (highest priority)   |
|    /etc/claude-code/settings.json        |
|    macOS MDM policies                    |
|    Remote API polling control            |
├──────────────────────────────────────────┤
```

```
│ 2. User Settings                          │
│    ~/.claude/settings.json                │
│    Global defaults                        │
├──────────────────────────────────────────┤
│ 3. Project Settings                       │
│    CLAUDE.md (project root)               │
│    .claude/CLAUDE.md                      │
│    .claude/rules/*.md                     │
├──────────────────────────────────────────┤
│ 4. Runtime Settings (lowest priority)     │
│    CLI flags (--permission-mode, etc.)    │
│    Environment variables                  │
│    (CLAUDE_CODE_* prefix)                 │
└──────────────────────────────────────────┘
```

8.2 Complete settings.json Structure

```
type GlobalConfig = {
  // ── General ──
  numStartups: number
  userID?: string
  installMethod?: 'local' | 'native' | 'global' | 'unknown'
  hasCompletedOnboarding?: boolean

  // ── Permissions ──
  theme: ThemeSetting
  permissions?: {
    defaultMode: PermissionMode
    rules?: {
      allow?: string[]     // e.g. 'Bash(prefix:rm)', 'FileEdit'
      deny?: string[]
      ask?: string[]
    }
  }

  // ── API & Auth ──
  primaryApiKey?: string
  customApiKeyResponses?: {
    approved?: string[]
    rejected?: string[]
  }

  // ── Environment ──
  env: { [key: string]: string }

  // ── Feature Toggles ──
  autoCompactEnabled: boolean
  verbose: boolean

  // ── Notifications ──
  preferredNotifChannel: 'system' | 'email' | 'slack'
```

```
  // ── OAuth ──
  oauthAccount?: {
    accountUuid: string
    emailAddress: string
    organizationUuid?: string
    displayName?: string
  }

  // ── MCP Servers ──
  mcpServers?: Record<string, McpServerConfig>

  // ── Hook Configuration ──
  hooks?: HookConfig[]

  // ── Project-Specific ──
  projects?: Record<string, ProjectConfig>
}
```

8.3 CLAUDE.md Memory System

Loading Hierarchy

```
1. Managed memory    /etc/claude-code/CLAUDE.md
2. User memory       ~/.claude/CLAUDE.md
3. Project memory    CLAUDE.md, .claude/CLAUDE.md, .claude/rules/*.md
4. Local memory      CLAUDE.local.md

Note: Files closer to the cwd take higher priority
Maximum character limit: 40,000
```

@include Directive

```
# CLAUDE.md
@path               # Relative path
@./rel/path         # Relative path
@~/home/path        # Home directory
@/abs/path          # Absolute path
```

Features:

- Circular reference detection and prevention
- Only loads text files
- Silently ignores missing files
- Character limit prevents oversized files

8.4 Environment Variables

Variable	Purpose
`CLAUDE_CODE_SIMPLE`	Simplified mode

ANTHROPIC_API_KEY	API key
CLAUDE_CODE_MAX_BUDGET	Maximum budget
CLAUDE_CODE_PERMISSION_MODE	Permission mode
CLAUDE_CODE_VERBOSE	Verbose output

Part 9: Context Management & Compaction

9.1 Token Budget System

```
// Core type
type TokenBudget = {
  maxContextTokens: number       // Model maximum context
  warningThreshold: number       // Warning threshold (80%)
  compactThreshold: number       // Compaction threshold
  estimatedCurrentTokens: number // Estimated current token count
}

// Budget check
calculateTokenWarningState(messages, limits) → {
  isNearLimit: boolean      // Near the limit
  isAtLimit: boolean        // At the limit
  shouldCompact: boolean    // Should compact
  estimatedTokens: number   // Estimated token count
}
```

9.2 Auto-Compaction Mechanism

```
Trigger conditions:
├── Manual: /compact command
├── Automatic: Context > 80% of limit
└── Reactive: After max_output_tokens recovery

Compaction flow:
┌──────────────────────────────────────┐
│ 1. Calculate compaction point        │
│    (compactionIndex)                 │
│    Preserve recent critical messages │
├──────────────────────────────────────┤
│ 2. Generate summary                  │
│    LLM summarizes old messages       │
├──────────────────────────────────────┤
│ 3. Build new message array           │
│    [system_prompt,                   │
│     attachment(summary),             │
│     remaining_messages]              │
└──────────────────────────────────────┘

Snip Compaction (Feature-gated):
├── Replace intermediate messages with minimal summaries
```

```
├── Retain only tool calls and results at boundaries
└── Dramatically reduce token consumption
```

9.3 Prompt Cache Optimization

Claude Code uses **multi-level prompt caching** to reduce redundant token consumption:

```
┌─ System Prompt Structure ──────────────────────┐
│                                                 │
│  ┌──────────────────────────────────────┐       │
│  │ Static portion (cacheable across     │       │
│  │ organizations)                       │       │
│  │ ├─ Base role instructions            │       │
│  │ ├─ Tool definitions                 │       │
│  │ ├─ General guidelines               │       │
│  │ └─ cache_control: ephemeral         │       │
│  ├──────── Dynamic boundary ────────────┤       │
│  │ __SYSTEM_PROMPT_DYNAMIC_BOUNDARY__  │       │
│  ├──────────────────────────────────────┤       │
│  │ Dynamic portion (varies per          │       │
│  │ session)                             │       │
│  │ ├─ User context (CLAUDE.md)          │       │
│  │ ├─ Git status                        │       │
│  │ ├─ Session-specific settings         │       │
│  │ └─ Feature tips                      │       │
│  └──────────────────────────────────────┘       │
│                                                 │
│  Cache effect: ~90% redundant token             │
│  reduction in long conversations                │
└─────────────────────────────────────────────────┘
```

9.4 Section Caching Mechanism

```
// Regular section: computed once, reused for the entire session
systemPromptSection(name, compute): SystemPromptSection

// Uncacheable section: recomputed every turn
DANGEROUS_uncachedSystemPromptSection(name, compute, reason)

// Cache is cleared on /clear and /compact
```

Part 10: Agent System

10.1 Agent Definition

```
type AgentDefinition = {
  agentType: string          // Agent type identifier
  name: string               // Display name
  description: string        // Description
  whenToUse: string          // Usage scenario description
```

```
  // Capability configuration
  tools?: string[]           // Allowed tools (whitelist)
  disallowedTools?: string[] // Forbidden tools (blacklist)

  // Persona settings
  getSystemPrompt(ctx?): string  // System prompt
  memory?: string                // Memory type

  // Execution configuration
  model?: string                 // Model to use
  thinkingConfig?: ThinkingConfig // Thinking configuration
}
```

10.2 Built-in Agent Types

Agent	Function	Tool Restrictions
general-purpose	General autonomous agent	All tools
Plan	Architecture planning agent	Read-only tools, no edit/write
Explore	Codebase exploration agent	Read-only search tools
claude-code-guide	Help/tutorial agent	Glob, Grep, Read, WebFetch, WebSearch
code-reviewer	Code review agent	Read-only tools

10.3 Agent Execution Model

```
Main Agent
  │
  ├── Agent(subagent_type: "Explore")
  │     └── Independent context, read-only tools
  │         Returns research results
  │
  ├── Agent(subagent_type: "general-purpose")
  │     └── Full context, all tools
  │         Autonomously completes complex tasks
  │
  └── Agent(isolation: "worktree")
        └── Git worktree isolation
            Independent repository copy
            Merge or clean up upon completion
```

10.4 Agent Prompt Best Practices

Prompt guidelines embedded in the source code:

> **Brief the agent like a smart colleague who just walked into the room** — they haven't seen this conversation, don't know what you've tried, and don't understand why this task matters.
>
> - Explain what you want to accomplish and why

- Describe what you've already learned or ruled out
- Provide enough context for the agent to make judgments
- If you need a brief response, say so explicitly

10.5 Custom Agents

Users can create custom agents in the `~/.claude/agents/` directory:

```
// ~/.claude/agents/security-reviewer.json
{
  "agentType": "security-reviewer",
  "name": "Security Reviewer",
  "description": "An agent specialized in reviewing code security",
  "whenToUse": "When a security review is needed",
  "tools": ["FileRead", "Glob", "Grep", "Bash"],
  "disallowedTools": ["FileEdit", "FileWrite"],
  "systemPrompt": "You are a security expert specializing in reviewing
}
```

Part 11: System Prompt Construction

11.1 Prompt Build Flow

```
buildEffectiveSystemPrompt({
  mainThreadAgentDefinition?: AgentDefinition
  toolUseContext: ToolUseContext
  customSystemPrompt?: string
  defaultSystemPrompt: string[]
  appendSystemPrompt?: string
  overrideSystemPrompt?: string
})
```

Priority chain:

1. **Override prompt** — Replaces everything (e.g., /loop mode)
2. **Coordinator mode prompt** — Multi-agent coordination
3. **Agent system prompt** — If mainThreadAgentDefinition is set
4. **Custom prompt** — User-specified via `--system-prompt`
5. **Default prompt** — Generated from `prompts.ts`

11.2 Default System Prompt Sections

Section	Purpose	Content
`getSimpleIntroSection()`	Role introduction	"You are an interactive agent..."
`getSimpleSystemSection()`	System description	Tools, permissions, labels, hooks, compaction
`getSimpleDoingTasksSection()`	Task guidance	Software engineering best practices, code style, security

```
// ├─ Streaming response handling
// ├─ Error classification
// └─ Retry coordination
```

12.2 Exponential Backoff Retry Strategy

```
Attempt 1: Execute immediately
Attempt 2: Wait 1s
Attempt 3: Wait 2s
Attempt 4: Wait 4s
Attempt 5: Wait 8s (max 5 retries)

Retryable errors:
├── 429 (Rate limit)
├── 500 (Server error)
├── 502, 503, 504 (Gateway errors)
└── Transient network errors

Non-retryable errors:
├── 401 (Authentication failure)
├── 403 (Insufficient permissions)
├── 400 (Bad request)
└── 413 (Payload too large)
```

12.3 max_output_tokens Recovery Mechanism

When the API returns `stop_reason: "max_tokens"`:

```
max_output_tokens detected
     │
     ▼
┌──────────────────────────┐
│ Increment recovery       │
│ counter                  │
│ maxOutputTokensRecovery  │
│ Count++                  │
├──────────────────────────┤
│ Upgrade token budget     │
│ Request higher output    │
│ limit                    │
├──────────────────────────┤
│ If too many recoveries:  │
│ Attempt reactive         │
│ compaction               │
│ hasAttemptedReactive     │
│ Compact = true           │
├──────────────────────────┤
│ Re-enter query loop      │
└──────────────────────────┘
```

Part 13: Build System & Compilation Optimization

13.1 Five-Stage Build Pipeline

```
Stage 1: Copy
   src/ → build-src/ (preserve original files)

Stage 2: Transform
   ├─ feature('X') → false (dead code marking)
   ├─ MACRO.VERSION → '2.1.88' (version injection)
   ├─ import from 'bun:bundle' → stubs (Bun features)
   └─ Remove type-only imports

Stage 3: Entry Wrapping
   Create build-src/entry.ts → points to src/entrypoints/cli.tsx

Stage 4: Iterative Stubbing + Bundling
   Round N:
   ├─ 1. esbuild bundles entry.ts
   ├─ 2. Collect missing modules
   ├─ 3. Create stubs for missing modules
   └─ 4. Repeat until successful

Stage 5: Output
   Generate final cli.js + cli.js.map
```

13.2 Feature Gate Compile-Time Elimination

```
// In source code
if (feature('DAEMON')) {
  require('./daemon/main.js')
}

// After compilation (public release)
if (false) {
  // Entire branch eliminated by esbuild (dead code elimination)
}
```

This mechanism causes **108 modules** to be eliminated in the public release.

13.3 Eliminated Internal Features (108 Modules)

Internal infrastructure (~70):

- `daemon/main.js` — Background monitor
- `coordinator/workerAgent.js` — Multi-agent coordination
- `assistant/` — KAIROS autonomous mode
- `bridge/peerSessions.js` — Remote control peer management
- `skillSearch/` — Remote skill discovery
- `sessionTranscript/sessionTranscript.js` — Classifier backend

Feature-gated tools (~20):

- `REPLTool` — Interactive REPL
- `MonitorTool` — MCP monitoring
- `WebBrowserTool` — Browser automation
- `WorkflowTool` — Workflow execution
- `SnipTool` — Context snipping
- `SleepTool` — Delay/scheduling

Part 14: Power User Tips (Beginner)

14.1 Installation & First Run

```
# Global install
npm install -g @anthropic-ai/claude-code

# Or run directly with npx (no install needed)
npx @anthropic-ai/claude-code

# Launch in a project directory
cd /path/to/your/project
claude

# Launch with an initial prompt
claude "Help me analyze this project's structure"

# Headless mode (for script integration)
claude -p "List all TODO comments" > todos.txt
```

14.2 Understanding the Interface

```
┌──────────────────────────────────────────────────────────┐
│ Claude Code v2.1.88                                      │
│                                                          │
│ 💬 Previous Claude response...                           │
│                                                          │
│ 🔧 Tool: FileRead (src/index.ts)                         │  ← Tool calls s
│ ✓ Tool complete                                           │
│                                                          │
│ 💬 Claude's latest response...                           │
│                                                          │
│ > Your input cursor here _                               │  ← Input area
│                                                          │
│ [Tokens: 12,450 / 200,000]  [Cost: $0.03]               │  ← Status bar
└──────────────────────────────────────────────────────────┘

Keyboard shortcuts:
├── Enter        → Send message (multi-line: Shift+Enter)
├── Ctrl+C       → Interrupt current operation
```

```
├── Ctrl+D       → Exit Claude Code
├── Esc          → Cancel current input
├── Up/Down      → Browse input history
└── Tab          → Autocomplete slash commands
```

14.3 Creating Your First CLAUDE.md

This is the first and most important step to improving efficiency. Source code analysis shows that CLAUDE.md content is injected into the system prompt on **every turn**, directly influencing the model's behavior.

```
# Project Description

## Tech Stack
- Next.js 14 + TypeScript + App Router
- Prisma + PostgreSQL
- Tailwind CSS + shadcn/ui

## Development Commands
- `pnpm dev` — Start dev server (port 3000)
- `pnpm test` — Run Vitest tests
- `pnpm lint` — ESLint check
- `pnpm build` — Production build

## Directory Structure
- src/app/ — Page routes (App Router)
- src/components/ — React components (using shadcn/ui)
- src/lib/ — Utility functions and shared logic
- src/server/ — Server-side logic (tRPC routers)
- prisma/ — Database schema and migrations

## Coding Conventions
- Use function components + Hooks; no class components
- State management with Zustand, not Redux
- All APIs via tRPC; do not write REST routes
- CSS via Tailwind; do not write custom CSS files

## Important Constraints
- Do not modify src/lib/legacy/ — legacy code, slated for removal next
- Tests must pass before committing
- All database changes must include migration files
- Environment variable templates are in .env.example
```

Why this works:

- The model does not need to spend tokens exploring the tech stack → saves 30%+ tokens
- Coding conventions are injected directly → reduces rework
- Constraints are stated up front → prevents accidental mistakes

14.4 Permission Rule Configuration

The most important configuration for first-time users — sensible permission rules make interactions much smoother:

```
{
  "permissions": {
    "rules": {
      "allow": [
        "FileRead",
        "FileEdit",
        "FileWrite",
        "Glob",
        "Grep",
        "Bash(prefix:npm test)",
        "Bash(prefix:npm run)",
        "Bash(prefix:pnpm )",
        "Bash(prefix:git status)",
        "Bash(prefix:git diff)",
        "Bash(prefix:git log)",
        "Bash(prefix:git branch)",
        "Bash(prefix:ls)",
        "Bash(prefix:cat)",
        "Bash(prefix:pwd)",
        "Bash(prefix:echo)",
        "Bash(prefix:node -e)",
        "Bash(prefix:npx tsc --noEmit)",
        "Bash(prefix:npx prisma)"
      ],
      "deny": [
        "Bash(prefix:rm -rf /)",
        "Bash(prefix:sudo)",
        "Bash(prefix:git push --force)",
        "Bash(prefix:DROP )",
        "Bash(prefix:TRUNCATE )"
      ]
    }
  }
}
```

Configuration principles:

Category	Strategy	Example
Read-only operations	Allow all	FileRead, Glob, Grep, git status
Common project commands	Allow by prefix	npm test, npm run, pnpm
File modifications	Allow (protected by Read)	FileEdit, FileWrite
Dangerous operations	Deny	rm -rf /, sudo, force push
Everything else	Default ask	Unmatched commands require confirmation

14.5 Complete Slash Command Quick Reference

Command	Function	Key Usage
`/help`	Show help	List all available commands
`/compact`	Compact context	Use when conversation gets too long; preserves key info
`/clear`	Clear history	Start fresh; clears all context
`/status`	Session status	View token usage and model info
`/cost`	Cost statistics	Cumulative API cost for this session
`/model`	Switch model	e.g. `/model sonnet` to switch to Sonnet
`/commit`	Git commit	Auto-generate commit message and commit
`/review`	Code review	Review changes on the current branch
`/mcp`	MCP management	Add/remove/list MCP servers
`/config`	Config management	View/modify settings.json
`/resume`	Resume session	Resume a previously interrupted conversation
`/tasks`	Task list	View Claude's task progress
`/permissions`	Permission management	View/modify permission rules
`/hooks`	Hook management	View/test configured hooks
`/doctor`	Diagnostic check	Check for environment configuration issues
`/fast`	Fast mode	Toggle Fast mode (same model, faster output)

14.6 Six Essential Usage Patterns

Pattern 1: Codebase Exploration

```
You: Help me understand this project's structure, focusing on API route

# Behind the scenes — tool call chain:
# Glob("**/*.ts") → discover file structure
# Grep("router|route|endpoint") → locate API definitions
# Read(key files) → deep understanding
# Generate structured overview
```

Pro tip: Giving a specific direction is 3x faster than open-ended exploration.

Pattern 2: Bug Location & Fix

```
You: Users get "Cannot read property 'email' of undefined" when submitt
    the error occurs at src/components/UserForm.tsx line 42

# Behind the scenes — execution chain:
# Read(UserForm.tsx) → locate the error line
# Grep("email") → trace the data flow
# Read(related API) → understand the full chain
```

```
# Edit(fix code) → apply the fix
# Bash("npm test") → verify the fix
```

Pro tip: Providing the error message and file location speeds up fixing by 5x.

Pattern 3: Feature Development

```
You: Add a user avatar upload feature. Requirements:
    - Support JPG/PNG, max 5MB
    - Upload to S3
    - Store URL in database
    - Frontend uses shadcn Upload component
```

Pro tip: Specifying the technology choices lets the model code directly, instead of asking you first.

Pattern 4: Code Refactoring

```
You: Unify all fetch calls in src/utils/api.ts to use axios instead,
    keep the interface signatures unchanged, and add unified error hand
```

Pattern 5: Writing Tests

```
You: Write unit tests for src/server/routers/user.ts,
    using Vitest + MSW to mock HTTP requests,
    covering both happy paths and error paths
```

Pattern 6: Code Review

```
You: /review

# Or a more targeted review
You: Review recent changes in the src/auth/ directory,
    focusing on security vulnerabilities and performance issues
```

14.7 Shell Integration Tips

```
# Run interactive commands inside Claude Code (! prefix)
You: ! gcloud auth login
You: ! docker-compose up -d
You: ! npx prisma studio

# Pipe output to Claude (headless mode)
git diff | claude -p "Review these changes"
cat error.log | claude -p "Analyze these error logs"
curl api.example.com/status | claude -p "Explain this API response"

# Write Claude output to a file
claude -p "Generate the contents of .github/workflows/ci.yml" > .github
```

```
# Switch between projects
You: Please switch the working directory to ../backend-service
```

14.8 Understanding Permission Prompts

When Claude Code needs to perform a potentially risky operation, a permission prompt appears:

```
Claude wants to run: rm -rf dist/ && npm run build

  [Y] Allow once       — Allow this time only
  [A] Always allow      — Remember and always allow this pattern
  [N] Deny             — Deny this time
  [D] Always deny       — Remember and always deny

Recommended strategy:
├── Build/test commands → Always allow
├── File deletion → Allow once (confirm, then allow once)
├── Unfamiliar commands → Deny (learn about it first)
└── Dangerous commands → Always deny (e.g. force push)
```

Part 15: Power User Tips (Intermediate)

15.1 CLAUDE.md Master Class

Layered Strategy: Global → Project → Rule Files

```
~/.claude/CLAUDE.md                    # Global preferences (shared acr
├── "I am a senior full-stack developer with 10 years of experience"
├── "Prefer functional programming and immutable data"
├── "Reply in English; keep technical terms as-is"
├── "Write code comments in English"
└── "Do not explain basic concepts; give solutions directly"

project-root/CLAUDE.md                 # Project configuration
├── Tech stack, dev commands, directory structure
├── @./docs/architecture.md            # Reference architecture doc
├── @./docs/api-spec.md               # Reference API spec
└── @./CONTRIBUTING.md                 # Reference contribution guide

.claude/rules/security.md              # Security rules
├── "All user input must be validated with zod"
├── "SQL queries must go through Prisma ORM only"
├── "Do not use eval(), innerHTML, or dangerouslySetInnerHTML"
└── "API routes must verify JWT tokens"

.claude/rules/style.md                 # Style rules
├── "React components use PascalCase"
├── "Utility functions use camelCase"
```

```
├── "Constants use UPPER_SNAKE_CASE"
├── "File names use kebab-case"
└── "Each file should not exceed 200 lines"

.claude/rules/testing.md               # Testing rules
├── "New features must have unit tests"
├── "Test files go in __tests__/ directories"
├── "Use describe/it structure"
├── "Mock external services, do not mock internal modules"
└── "Test coverage must not drop below 80%"
```

CLAUDE.md Templates for Different Project Types

Frontend React project:

```
# Frontend Project

## Stack: React 18 + TypeScript + Vite + Zustand + React Query

## Commands
- `pnpm dev` — Dev server on :5173
- `pnpm test` — Vitest
- `pnpm storybook` — Component showcase

## Conventions
- Prefer server components where possible
- Use React Query for all API calls, no raw fetch
- All forms use react-hook-form + zod
- Components: src/components/{feature}/{ComponentName}.tsx
- Hooks: src/hooks/use{HookName}.ts
- Never use `any` type — prefer `unknown` + type guard

## Key APIs
- Auth: /api/auth/* (NextAuth)
- Data: /api/trpc/* (tRPC)
```

Backend Python project:

```
# Backend Service

## Stack: Python 3.12 + FastAPI + SQLAlchemy 2.0 + Alembic + pytest

## Commands
- `uv run uvicorn app.main:app --reload` — Dev server
- `uv run pytest` — Tests
- `uv run alembic upgrade head` — Run migrations
- `uv run mypy .` — Type check

## Conventions
- Use async/await everywhere
- Type hints on all functions
```

```
- Pydantic models for request/response
- One router file per resource in app/routers/
- Business logic in app/services/, not in routers
- Database models in app/models/

## Important
- Never use raw SQL strings — always use SQLAlchemy
- All endpoints need OpenAPI docs (docstring + response_model)
- Environment variables via app/config.py (pydantic-settings)
```

Infrastructure / DevOps project:

```
# Infrastructure

## Stack: Terraform + AWS + Docker + GitHub Actions

## Commands
- `terraform plan` — Preview changes
- `terraform apply` — Apply changes (DANGEROUS)
- `docker compose up -d` — Local stack

## Rules
- NEVER run terraform apply without showing me the plan first
- NEVER modify production tfvars files
- All resources must have proper tags
- Use modules from modules/ directory
- State is in S3 — do not modify backend config
```

Advanced @include Usage

```
# CLAUDE.md

## Core documentation (auto-injected into every conversation turn)
@./docs/architecture.md
@./docs/api-spec.md
@./CONVENTIONS.md

## Conditional references
@./docs/database-schema.sql
@./openapi.yaml

## Notes
# @include file contents count toward the 40,000-character limit
# Prefer referencing concise documents; do not reference entire READMEs
# If a file does not exist, it will be silently ignored
```

15.2 Prompt Engineering Compendium (15 Practical Patterns)

Effective prompt patterns derived from system prompt source code analysis:

Pattern 1: Precise Targeting

```
BAD:  "Help me fix this file"
BAD:  "The login feature has a bug"
GOOD: "Modify the getUser function in src/api/users.ts:42,
       change the database query from findFirst to findUnique,
       because the id field is unique"
```

Pattern 2: Context Preloading

```
You: First read the following files and understand the data flow before
    - src/models/user.ts
    - src/server/routers/user.ts
    - src/components/UserProfile.tsx
    Then tell me, if I want to add a "user address" field,
    which files need to be changed and in what order?
```

Rationale: The Read tool in the source code is far more efficient than Bash(cat) and populates the file cache.

Pattern 3: Step-by-Step Confirmation

```
You: I need to migrate the database from MySQL to PostgreSQL.
    Please follow these steps, waiting for my confirmation after each o
    1. Analyze all uses of MySQL-specific syntax
    2. List the files that need to be modified
    3. Modify the ORM configuration
    4. Modify data type mappings
    5. Modify raw SQL queries
    6. Update tests
```

Pattern 4: Role Assignment

```
You: You are now a security audit expert. Perform a comprehensive
    security review of the src/auth/ directory, checking against
    each item in the OWASP Top 10, and assign a severity rating
    (Critical/High/Medium/Low)
```

Pattern 5: Comparative Analysis

```
You: Compare the implementations of src/services/old-payment.ts and
    src/services/new-payment.ts.
    Does the new version fully cover all functionality of the old versi
    Are there any edge cases that were missed?
```

Pattern 6: Reverse Engineering

```
You: Read src/lib/encryption.ts, explain the encryption flow,
    draw a data flow diagram, and annotate the input/output types at ea
```

Pattern 7: Constraint-Driven

```
You: Implement a cache module with these constraints:
    - Zero external dependencies
    - TTL support
    - LRU eviction support
    - Thread-safe
    - Single file, no more than 100 lines
```

Pattern 8: Test-Driven Development (TDD)

```
You: Implement an email validation function using TDD:
    1. Write the test cases first (normal, boundary, error)
    2. Run tests to confirm they all fail
    3. Implement the minimum code to make tests pass
    4. Refactor
```

Pattern 9: Batch Operations

```
You: Across the entire project:
    1. Replace all console.log with logger.debug
    2. Replace all console.error with logger.error
    3. Add import { logger } from '@/lib/logger' at the top of each fil
    4. Make sure nothing is missed, and run tests to verify
```

Pattern 10: Documentation Generation

```
You: Generate Markdown documentation for all API endpoints in
    src/server/routers/, in this format:
    ## [Endpoint Name]
    - Method: GET/POST/...
    - Path: /api/...
    - Parameters: {...}
    - Response: {...}
    - Example: curl ...
```

Pattern 11: Progressive Refactoring

```
You: Refactor src/utils/helpers.ts into smaller modules,
    but ensure backward compatibility:
    - Put refactored code into src/utils/ subdirectories
    - Change the original file to re-export
    - Run tests to make sure nothing breaks
```

Pattern 12: Error Handling Enhancement

```
You: Review all routes under src/api/ and find places with missing erro
    - Async operations without try-catch
    - Endpoints without input validation
    - Places not returning proper error codes
    Fix all issues, consistently using the project's AppError class
```

Pattern 13: Performance Analysis

```
You: Analyze performance issues in src/pages/dashboard.tsx:
    - Find unnecessary re-renders
    - Check for N+1 queries
    - Check for memory leak risks
    - Suggest usage of React.memo / useMemo / useCallback
    Give concrete code changes, not just advice
```

Pattern 14: Migration Scripts

```
You: Write a data migration script scripts/migrate-user-roles.ts:
    - Migrate from user.role (string) to a user_roles table (many-to-ma
    - Support rollback
    - Include progress output
    - Handle interrupted resume
    - Dry-run mode
```

Pattern 15: Parallel Agent Delegation

```
You: Execute the following independent tasks in parallel:
    1. [Explore Agent] Analyze all third-party API integration points i
    2. [General Agent] Write unit tests for UserService
    3. [Explore Agent] List all environment variables and their purpose

# Source code shows Claude Code will automatically:
# - Create an independent sub-agent for each task
# - Select the best agent type based on the task (Explore for read-only
# - Execute in parallel, aggregate results
```

15.3 Agent Parallel Work in Detail

Understanding Agent Type Selection

```
Agent types and their use cases (from source code analysis):

general-purpose     → Complex tasks requiring read/write operations
                      Has all tools, most versatile

Explore             → Codebase search and analysis
                      Read-only tools only, faster
                      Best for "find", "analyze", "understand"

Plan                → Designing implementation plans
                      Read-only + cannot edit/write
                      Best for "how to implement", "design a plan"

claude-code-guide   → Help with Claude Code itself
                      Glob, Grep, Read, WebFetch, WebSearch
```

Efficient Parallel Patterns

```
You: I need to release a new version. Please complete in parallel:
    1. Run all tests and report results
    2. Check whether there are uncommitted files
    3. Review all commits since the last release
    4. Check whether the package.json version has been updated

# Claude Code will launch 4 agents in parallel
# All results are aggregated as each agent completes
```

Worktree Isolation Experiments

```
You: Please try upgrading React to v19 in an isolated environment,
    test whether all components are compatible, and do not affect the c

# Claude Code will:
# 1. git worktree add → create an independent repository copy
# 2. Execute all modifications and tests in the worktree
# 3. Success → return the worktree path and branch name for you to merg
# 4. Failure → automatically clean up, report which components are inco
```

15.4 MCP Server Practical Configuration

Common MCP Server Combinations for Developers

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
    },
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": { "DATABASE_URL": "postgresql://localhost:5432/mydb" }
    },
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/path/
    },
    "memory": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-memory"]
    },
    "brave-search": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-brave-search"],
      "env": { "BRAVE_API_KEY": "${BRAVE_API_KEY}" }
```

```
    },
    "puppeteer": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-puppeteer"]
    }
  }
}
```

MCP Server Use Cases

MCP Server	Unlocked Capabilities	Example Usage
GitHub	PR/Issue management	"Create a PR", "View Issue #42"
PostgreSQL	Direct database queries	"Show recently registered users", "Analyze table structure"
Filesystem	Access files outside the project	"Read ~/Documents/spec.pdf"
Memory	Persistent memory across sessions	"Remember this architecture decision"
Brave Search	Search for the latest information	"Search for React 19 breaking changes"
Puppeteer	Browser automation	"Take a screenshot of the page", "Test form submission"
Slack	Send notifications	"Post the test results to the #dev channel"
Linear/Jira	Task management	"Update the Linear ticket status"

15.5 Hook Automation Workflow Library

Hook 1: Auto-Format (Format on Save)

```
{
  "event": "PostToolUse",
  "matcher": "FileEdit|FileWrite",
  "handler": {
    "type": "script",
    "command": "npx prettier --write \"$TOOL_INPUT_FILE_PATH\" 2>/dev/n
  }
}
```

Hook 2: Auto-Lint Check

```
{
  "event": "PostToolUse",
  "matcher": "FileEdit",
  "handler": {
    "type": "script",
    "command": "npx eslint \"$TOOL_INPUT_FILE_PATH\" --fix 2>&1 | tail
  }
```

```
}
```

Hook 3: Sensitive File Protection

```
{
  "event": "PreToolUse",
  "matcher": "FileEdit|FileWrite",
  "handler": {
    "type": "javascript",
    "code": "module.exports = async ({input}) => { const p = input.file
  }
}
```

Hook 4: Commit Message Convention Check

```
{
  "event": "PreToolUse",
  "matcher": "Bash",
  "handler": {
    "type": "javascript",
    "code": "module.exports = async ({input}) => { const cmd = input.co
  }
}
```

Hook 5: TypeScript Type Check Gate

```
{
  "event": "PostToolUse",
  "matcher": "FileEdit",
  "handler": {
    "type": "script",
    "command": "if echo \"$TOOL_INPUT_FILE_PATH\" | grep -qE '\\.(ts|ts
  }
}
```

Hook 6: Auto-Run Related Tests

```
{
  "event": "PostToolUse",
  "matcher": "FileEdit",
  "handler": {
    "type": "script",
    "command": "TEST_FILE=$(echo \"$TOOL_INPUT_FILE_PATH\" | sed 's/\\.
  }
}
```

15.6 Advanced Session Management

Session Recovery and Continuation

```
# List recent sessions
claude --resume

# Resume a specific session
claude --resume [session-id]

# Tip: name your sessions
You: Please name the current session "user system refactor"
# You can then search by name to resume later
```

Context Management Strategy

```
Context management timeline for long tasks:

Phase 1 (0-30% context): Free exploration and coding
        ↓
Phase 2 (30-60%): Start using context strategically
        ├── Use Grep instead of reading full files
        ├── Give precise line number ranges
        └── Use concise instructions
        ↓
Phase 3 (60-80%): Active management
        ├── /compact to compress non-critical history
        ├── Split large tasks into new sessions
        └── Use Agent to delegate independent sub-tasks
        ↓
Phase 4 (80%+): Auto-Compact triggers
        └── System compacts automatically; early details may be lost

Best practice: Proactively /compact during Phases 2-3
```

15.7 Git Workflow Integration

Smart Commit

```
You: /commit

# Claude Code will:
# 1. git status to see changes
# 2. git diff to analyze all modifications
# 3. git log to reference historical style
# 4. Generate a well-formatted commit message
# 5. Execute git commit

# Tip: giving a direct instruction is more efficient
You: Commit all recent changes; use conventional commits format for the
```

Code Review Workflow

```
You: /review
```

```
# Or a more precise review
You: Review all changes on the current branch relative to main, focusin
    1. Security vulnerabilities
    2. Performance issues
    3. Code style consistency
    4. Test coverage
    Give specific improvement suggestions with code examples

# Review a PR
You: Review the code in GitHub PR #123
```

Branch Management

```
You: Create a new branch feature/user-avatar from main,
    then implement the user avatar feature

You: Does the current branch have conflicts with main? If so, help me r

You: Squash the 3 commits on the current branch into one,
    and generate a clear commit message
```

15.8 Multi-File Refactoring Patterns

```
You: Migrate the project from CommonJS (require/module.exports)
    to ESM (import/export):
    1. First analyze how many files need to be changed
    2. List the modification plan for my confirmation
    3. Modify in dependency order (leaf files first)
    4. Run tests after each batch of file changes
    5. Update package.json and tsconfig.json

# Claude Code will use Glob to find all files
# Use Grep to analyze require/module.exports usage
# Modify in topological order
# Use Bash to run tests verifying each step
```

Part 16: Power User Tips (Advanced)

16.1 Deep Understanding of the Permission System

The **permission decision chain** discovered in the source code:

```
Tool call → Allow Rules → Deny Rules → Ask Rules
         → Permission Mode → YOLO Classifier → User Prompt

Decision reason types (PermissionDecisionReason):
├── classifier   — Auto mode classifier decision
├── hook         — Decision returned by a hook
```

```
├── rule         — Permission rule match
├── mode         — Permission mode decision
├── safetyCheck  — Safety check
├── sandboxOverride — Sandbox override
├── workingDir   — Working directory check
└── asyncAgent   — Async agent
```

Precise configuration strategy:

```
{
  "permissions": {
    "rules": {
      "allow": [
        "Bash(prefix:npm )",
        "Bash(prefix:pnpm )",
        "Bash(prefix:yarn )",
        "Bash(prefix:node )",
        "Bash(prefix:npx )",
        "Bash(prefix:git status)",
        "Bash(prefix:git diff)",
        "Bash(prefix:git log)",
        "Bash(prefix:git branch)",
        "Bash(prefix:git stash)",
        "Bash(prefix:docker compose)",
        "Bash(prefix:docker ps)",
        "Bash(prefix:curl )",
        "Bash(prefix:cat )",
        "Bash(prefix:ls )",
        "Bash(prefix:find )",
        "Bash(prefix:wc )",
        "Bash(prefix:head )",
        "Bash(prefix:tail )",
        "Bash(prefix:grep )",
        "Bash(prefix:echo )",
        "Bash(prefix:pwd)",
        "Bash(prefix:which )",
        "Bash(prefix:env | grep)",
        "FileRead",
        "FileEdit",
        "FileWrite",
        "Glob",
        "Grep",
        "Agent",
        "NotebookEdit",
        "WebFetch",
        "WebSearch"
      ],
      "deny": [
        "Bash(prefix:rm -rf /)",
        "Bash(prefix:sudo )",
        "Bash(prefix:chmod 777)",
```

```
        "Bash(prefix:git push --force)",
        "Bash(prefix:git reset --hard)",
        "Bash(prefix:DROP )",
        "Bash(prefix:TRUNCATE )",
        "Bash(prefix:shutdown)",
        "Bash(prefix:reboot)",
        "Bash(prefix:mkfs)"
      ]
    }
  }
}
```

16.2 Fine-Grained Token Budget Management

Cost Estimation Formula (Derived from Source Code)

```
Per-conversation cost ≈ System prompt tokens × number of turns × input
                      + Total output tokens × output price
                      + Tool call result tokens × input price

Optimization levers (ranked by impact):
1. Reduce system prompt size → keep CLAUDE.md concise
2. Reduce turns → precise instructions + say everything at once
3. Reduce tool calls → provide file paths to avoid searches
4. Model selection → use Haiku for simple tasks
5. Compact promptly → /compact to avoid redundancy
```

Cost Comparison by Task Type

Task Type	Estimated Turns	Estimated Cost	Optimization Tips
Simple bug fix	3-5	$0.02-0.05	Provide file path and line number
Feature development	10-20	$0.10-0.30	Step-by-step confirmation, preset in CLAUDE.md
Code review	5-10	$0.05-0.15	Use the built-in /review command
Full project refactor	30-60	$0.50-2.00	Split into sub-tasks, use parallel agents
Documentation generation	5-10	$0.05-0.10	Use templated output

Practical Cost-Saving Tips

```
Tip 1: Model Switching
├── Complex architecture design → Opus (strongest reasoning)
├── Daily coding → Sonnet (best cost-performance ratio)
├── Simple changes/formatting → Haiku (cheapest)
└── Switch command: /model sonnet
```

```
Tip 2: Reduce Wasted Tokens
├── Say "Don't explain, just change the code"
├── Provide file paths instead of making it search
├── Give all context at once, instead of asking incrementally
└── Use Agent in parallel, not sequentially

Tip 3: Smart Compact
├── /compact immediately after completing a feature
├── /compact when switching topics
├── /compact when responses start getting slow
└── Do not wait for Auto-Compact (by then many tokens are already waste
```

16.3 Building Complex Hook Pipelines

AI-Driven Code Review Gate

```
{
  "event": "PreToolUse",
  "matcher": "FileEdit",
  "handler": {
    "type": "agent",
    "prompt": "Review the upcoming file edit. Check for:\n1. Security v
    "timeout": 30,
    "model": "claude-haiku-4-5-20251001"
  }
}
```

Fully Automated Local CI Pipeline

```
{
  "hooks": [
    {
      "event": "PostToolUse",
      "matcher": "FileEdit|FileWrite",
      "handler": {
        "type": "script",
        "command": "bash -c 'F=\"$TOOL_INPUT_FILE_PATH\"; npx prettier
      }
    },
    {
      "event": "PostToolUse",
      "matcher": "FileEdit",
      "handler": {
        "type": "script",
        "command": "bash -c 'F=\"$TOOL_INPUT_FILE_PATH\"; TF=$(echo \"$
      }
    },
    {
      "event": "PostToolUse",
      "matcher": "Bash",
```

```
      "handler": {
        "type": "javascript",
        "code": "module.exports = async ({input}) => { const c = input.
      }
    }
  ]
}
```

16.4 SDK / Headless Mode Integration

Calling Claude Code from Scripts

```
// script.ts — batch code analysis
import Anthropic from "@anthropic-ai/sdk";

// Claude Code SDK mode
const client = new Anthropic();

// You can also use the CLI's -p mode directly
import { execSync } from "child_process";

// Analyze security of multiple files
const files = ["src/auth.ts", "src/api.ts", "src/db.ts"];
for (const file of files) {
  const result = execSync(
    `claude -p "Analyze security risks in ${file}, output in JSON forma
    { encoding: "utf-8" }
  );
  console.log(`${file}: ${result}`);
}
```

CI/CD Integration

```
# .github/workflows/claude-review.yml
name: Claude Code Review
on: [pull_request]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Install Claude Code
        run: npm install -g @anthropic-ai/claude-code

      - name: Run Review
        env:
```

```
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
        run: |
          DIFF=$(git diff origin/main...HEAD)
          claude -p "Review the following diff, find bugs and security
            --output-format json > review.json

      - name: Comment on PR
        uses: actions/github-script@v7
        with:
          script: |
            const review = require('./review.json');
            github.rest.issues.createComment({
              owner: context.repo.owner,
              repo: context.repo.repo,
              issue_number: context.issue.number,
              body: review.result
            });
```

16.5 Creating Custom Agents

```
// ~/.claude/agents/security-auditor.json
{
  "agentType": "security-auditor",
  "name": "Security Auditor",
  "description": "An agent specialized in code security auditing",
  "whenToUse": "When you need security reviews, vulnerability detection
  "tools": ["FileRead", "Glob", "Grep", "Bash", "WebSearch"],
  "disallowedTools": ["FileEdit", "FileWrite"],
  "systemPrompt": "You are a senior security engineer focused on:\n1. O
}

// ~/.claude/agents/db-expert.json
{
  "agentType": "db-expert",
  "name": "Database Expert",
  "description": "Database design and query optimization expert",
  "whenToUse": "When you need database design, query optimization, or m
  "tools": ["FileRead", "Glob", "Grep", "Bash"],
  "systemPrompt": "You are a PostgreSQL database expert skilled in:\n1.
}

// ~/.claude/agents/doc-writer.json
{
  "agentType": "doc-writer",
  "name": "Documentation Writer",
  "description": "Technical documentation and API documentation writing
  "whenToUse": "When you need to write documentation, READMEs, or API d
  "tools": ["FileRead", "FileWrite", "Glob", "Grep"],
  "systemPrompt": "You are a technical documentation expert. Your outpu
}
```

Usage:

```
You: Use the security-auditor agent to review the entire src/auth/ dire

You: Use the db-expert agent to optimize this slow query:
    SELECT * FROM orders JOIN users ON ...
```

16.6 Performance Optimization Checklist

#	Tip	Rationale (Source Code Basis)	Expected Impact
1	Write a good CLAUDE.md	Injected into system prompt, reduces exploration	Tokens -30%
2	Provide file paths	Skips Glob searches	2x faster responses
3	Provide line number ranges	Read(offset, limit) reduces data read	Tokens -50%/file
4	Give all info at once	Reduces back-and-forth turns	Turns -60%
5	Use parallel Agents	Sub-agents have independent contexts	3x speed
6	/compact promptly	Reduces per-turn token consumption	2x endurance for long conversations
7	Configure allow rules	Reduces permission interaction waits	+50% fluency
8	Use Grep instead of Read	Fetches only relevant lines	Tokens -80%/search
9	Use Haiku for simple tasks	Model downgrade	Cost -90%
10	Batch operations at once	Reduces round trips	+200% efficiency

16.7 Complete Debugging & Diagnostics Guide

```
# ──────── Session Info ────────
/status          # Token usage, model, session ID
/cost            # Cumulative cost for this session
/config          # Current configuration details

# ──────── Environment Diagnostics ────────
/doctor          # Check environment, dependencies, configuration issue

# ──────── Context Diagnostics ────────
/compact         # View context usage + compress
                 # analyzeContext.ts in the source computes:
                 # - Message token count
                 # - System prompt token count
                 # - Remaining budget
                 # - Whether compaction is needed
```

```
# ──────── Verbose Mode ────────
# Set in settings.json
{ "verbose": true }
# This will show:
# - Token consumption for each API call
# - Detailed tool call parameters
# - Permission decision reasons
# - Hook execution results

# ──────── Quick Troubleshooting ────────

Problem: Responses getting slower and slower
Cause:   Context approaching the limit
Fix:     /compact or /clear and start fresh

Problem: Permission prompts too frequent
Cause:   Not enough allow rules
Fix:     Add commonly used commands to the allow rules

Problem: Claude forgot earlier discussion
Cause:   Auto-Compact compressed early context
Fix:     Write key information into CLAUDE.md or re-state it

Problem: MCP server connection failed
Cause:   Config error or process not started
Fix:     /mcp to check status, verify command paths

Problem: File edit results are wrong
Cause:   old_string is not unique or has hidden characters
Fix:     Provide more context to make old_string unique

Problem: Sub-agent not producing expected results
Cause:   Prompt too vague
Fix:     Give specific file paths, expected outcomes, and constraints
```

16.8 Multi-Project Coordination

```
# Switch projects within the same session
You: Switch to the ../api-service directory

# Use addDir to add additional working directories
You: /addDir ../shared-types
# Now Claude Code can access both projects simultaneously

# Cross-project operation example
You: I modified the User type in shared-types/src/user.ts.
    Please check everywhere in api-service and web-app that uses the Us
    and see which ones need to be updated

# Usage with microservice architectures
```

```
You: Analyze the API call relationships between api-gateway, user-servi
    and order-service, and draw a dependency graph
```

Part 17: Hidden Internal Features & Feature Gates

17.1 Feature Gate Mechanism

The source code uses `feature('FEATURE_NAME')` for compile-time conditional compilation:

```
// In public release: feature('X') → false
// In internal builds: feature('X') → true (evaluated at Bun compile ti
```

17.2 Known Feature Gate List

Feature Gate	Function	Status
`DAEMON`	Background daemon	Internal
`BRIDGE_MODE`	Remote control bridge	Internal
`BG_SESSIONS`	Background sessions	Internal
`COORDINATOR_MODE`	Multi-agent coordination	Internal
`KAIROS`	Fully autonomous assistant mode	Internal
`PROACTIVE`	Proactive agent	Internal
`TRANSCRIPT_CLASSIFIER`	Classifier	Internal
`AGENT_TRIGGERS`	Scheduled triggers	Partially open
`AGENT_TRIGGERS_REMOTE`	Remote triggers	Internal
`DIRECT_CONNECT`	Direct connection	Internal
`SSH_REMOTE`	SSH remote	Internal
`HISTORY_SNIP`	History snipping	Internal
`BREAK_CACHE_COMMAND`	Cache clearing command	Internal
`CHICAGO_MCP`	Computer Use MCP	Internal
`DUMP_SYSTEM_PROMPT`	Dump system prompt	Internal
`WORKFLOW_SCRIPTS`	Workflow scripts	Internal

17.3 Undisclosed Internal Tools

Tool	Function	Restriction
`REPLTool`	Interactive REPL	ANT-only
`MonitorTool`	MCP monitoring	Feature-gated
`WebBrowserTool`	Browser automation	Feature-gated
`WorkflowTool`	Workflow execution	Feature-gated
`SnipTool`	Context snipping	Feature-gated

`SleepTool`	Delay/scheduling	Feature-gated
`TerminalTool`	Terminal control	Feature-gated
`TeamCreateTool`	Team creation	Feature-gated
`TeamDeleteTool`	Team deletion	Feature-gated

17.4 KAIROS Autonomous Mode

KAIROS system information extracted from the source code:

```
KAIROS (Fully Autonomous Assistant Mode):
├── Always-running background AI agent
├── Push notification support
├── GitHub PR webhook integration
├── <tick> heartbeat mechanism — periodic autonomous execution
├── Push-to-talk voice mode
├── Proactive tools (SleepTool, CronTools)
└── Standalone assistant/ module system
```

17.5 Coordinator Multi-Agent Mode

```
Coordinator Mode:
├── Maintains independent conversation context for each agent
├── Dispatches work based on best-fit matching
├── Aggregates multi-agent results
├── workerAgent.js — Worker agent
└── coordinatorMode.js — Coordinator core
```

Part 18: Architectural Design Patterns Summary

18.1 Async Generator Streaming

```
// Core pattern: streaming via async generators
async function* query(params): AsyncGenerator<StreamEvent | Message> {
  // Yield individual events
  // Allows consumers to respond to partial results in real time
  // Resource cleanup via return()
}

// Consumption pattern
for await (const event of query(params)) {
  if (event.type === 'tool_call') { /* update UI */ }
  if (event.type === 'message') { /* display text */ }
}
```

18.2 Feature Gate Compile-Time Elimination

```
// Compile-time conditional compilation + dead code elimination
```

```
if (feature('FEATURE_NAME')) {
  // Bun: included when flag=true, DCE'd when flag=false
  // esbuild: feature() → false, entire branch eliminated
  require('./feature-module.js')
}
```

18.3 Lazy Loading to Break Circular Dependencies

```
// Avoid circular module references
const getTeammateUtils = () =>
  require('./utils/teammate.js') as typeof import('./utils/teammate.js'

// Only loaded when actually called
const utils = getTeammateUtils()
```

18.4 Memoization for Expensive Operations

```
export const getSystemContext = memoize(async () => {
  // Cached for the entire session
  // Cleared on settings changes
})
```

18.5 Discriminated Unions for Type Safety

```
type Message =
  | { type: 'user'; content: string }
  | { type: 'assistant'; message: { content: ContentBlockParam[] } }
  | { type: 'attachment'; metadata: { name: string } }

// Compile-time exhaustiveness checking
if (message.type === 'user') { ... }
else if (message.type === 'assistant') { ... }
// TypeScript ensures all types are handled
```

18.6 Parallel Prefetching for Startup Optimization

```
// Parallel prefetch pattern in main.tsx
startMdmRawRead()        // Parallel 1: MDM subprocess
startKeychainPrefetch()  // Parallel 2: Keychain read

// Executes in parallel with module loading (~135ms)
// Results awaited on demand when needed
```

18.7 Dual-Track State Management

```
React State (AppStateStore):
├── Used by interactive UI
├── Lifecycle managed by React
└── For REPL mode
```

```
Bootstrap State (bootstrap/state.ts):
├── Global mutable state
├── Available before React renders
└── For SDK/headless mode
```

Part 21: Practical Workflow Recipes

This chapter provides 10 complete real-world scenarios, each with specific prompts, expected behavior, and best practices.

Recipe 1: Full-Stack Feature Development (User Notification System)

```
You: I need to implement a user notification system, including:
    - Database: notifications table (id, user_id, type, title, body, re
    - API: GET /notifications, POST /notifications/read/:id, GET /notif
    - Frontend: notification bell icon + dropdown list + unread count b
    - Real-time: WebSocket push for new notifications

    Please implement in this order:
    1. Database migration
    2. Backend API
    3. WebSocket handling
    4. Frontend components
    5. Integration tests
    Run tests after each step to confirm

Expected behavior:
├── Claude reads existing code to understand the architecture
├── Creates Prisma migration
├── Implements tRPC router
├── Adds WebSocket handler
├── Creates React components
├── Writes tests
└── Runs all tests to confirm they pass
```

Recipe 2: Bug Investigation & Fix (Production Issue)

```
You: Production is reporting the following error, approximately 50 time

    Error: Connection pool exhausted
    at PostgresPool.acquire (src/db/pool.ts:45)
    at UserRepository.findById (src/repos/user.ts:23)
    at AuthMiddleware.verify (src/middleware/auth.ts:67)

    Please:
    1. Analyze possible root causes (do not rush to fix)
    2. Check connection pool configuration and usage patterns
```

```
    3. Search all database connection usage to find leak points
    4. Propose a fix plan for my confirmation
    5. Implement the fix once confirmed
    6. Add monitoring/alerting code to prevent recurrence

Expected behavior:
├── Read pool.ts:45 and surrounding code
├── Grep to search all database connection usage
├── Analyze connection release patterns (are connections left unclosed?
├── Check error handling in transactions (is release in finally?)
├── Propose a plan (wait for confirmation)
├── Fix the code
└── Add connection pool monitoring middleware
```

Recipe 3: Legacy Code Modernization

```
You: The src/legacy/ directory contains Express.js code written in 2019
    I need to gradually migrate it to the existing Next.js App Router a

    Rules:
    - Migrate only one endpoint at a time
    - Keep old code functional until new code passes tests
    - Maintain backward compatibility (same request/response format)
    - Use Prisma to replace raw SQL

    Please first analyze what endpoints exist in src/legacy/,
    sort them by complexity, and start migrating from the simplest.

Expected behavior:
├── Glob and Grep to analyze the legacy directory
├── List all endpoints with their complexity
├── Starting from the simplest:
│   ├── Read the old implementation
│   ├── Create new App Router version
│   ├── Convert raw SQL to Prisma
│   ├── Write tests
│   ├── Run tests to confirm
│   └── Report completion, continue to next
└── Each endpoint is completed and verified independently
```

Recipe 4: API Design & Implementation

```
You: Design a RESTful API for a blog system. Requirements:
    - Resources: posts, comments, tags, users
    - Authentication: JWT
    - Pagination: cursor-based
    - Filtering: support multiple fields
    - Sorting: support multiple fields

    Please design the API documentation (OpenAPI format) first,
```

```
    then implement only after I confirm.

Expected behavior:
├── Generate OpenAPI YAML specification
├── Wait for user confirmation
├── Implement database schema
├── Implement each endpoint
├── Add middleware (auth, pagination, validation)
├── Write integration tests
└── Generate API documentation
```

Recipe 5: Database Schema Changes

```
You: I need to change the user system from single-role to multi-role:
    - Current: users table has a role string field
    - Target: roles table + user_roles join table
    - Requirements:
      1. Create migration script
      2. Data migration (preserve existing role data)
      3. Update all code that uses user.role
      4. Update API response format
      5. Support rollback

    Important: First generate a dry-run migration plan,
    listing all files that will be affected

Expected behavior:
├── Analyze all usage points of user.role
├── Generate impact analysis report
├── Create migration file (including data migration)
├── Modify ORM models
├── Update all referencing code
├── Update the API layer
├── Write tests
└── Verify that rollback works
```

Recipe 6: Performance Optimization

```
You: The dashboard page takes 8 seconds to load, which is too slow.
    The page component is src/pages/dashboard/index.tsx

    Please analyze and optimize completely:
    1. Check API calls (are there serial requests that should be parall
    2. Check database queries (N+1, missing indexes)
    3. Check frontend rendering (unnecessary re-renders)
    4. Check data size (are we fetching unneeded fields?)
    5. Suggest caching strategies

    Give the expected performance improvement for each optimization
```

```
Expected behavior:
├── Read the dashboard component code
├── Trace all API calls → look for serial/waterfall requests
├── Trace SQL queries → look for N+1 and full table scans
├── Analyze React rendering → look for unnecessary state changes
├── Propose optimization plan (each with expected benefit)
├── Implement optimizations
├── Add performance marks for verification
└── Run tests to confirm functionality is unaffected
```

Recipe 7: Security Audit

```
You: Perform a security audit of the entire project, checking against O
    A01 - Broken Access Control
    A02 - Cryptographic Failures
    A03 - Injection
    A04 - Insecure Design
    A05 - Security Misconfiguration
    A06 - Vulnerable and Outdated Components
    A07 - Identification and Authentication Failures
    A08 - Software and Data Integrity Failures
    A09 - Security Logging and Monitoring Failures
    A10 - Server-Side Request Forgery

    For each finding, provide: severity, file location, remediation

Recommended combo: use the security-auditor custom Agent
You: Use the security-auditor Agent to review these directories in para
    1. src/auth/
    2. src/api/
    3. src/middleware/
```

Recipe 8: Batch Documentation Generation

```
You: Generate the following documentation for the project:
    1. README.md — Project intro, installation, usage
    2. API.md — All API endpoint docs (auto-generated from code)
    3. ARCHITECTURE.md — Architecture overview + data flow diagram
    4. CONTRIBUTING.md — Contribution guide
    5. CHANGELOG.md — Changelog generated from git log

    Generate an outline for each document first for my confirmation

Expected behavior:
├── Glob + Grep to analyze project structure
├── Generate outline for each document (wait for confirmation)
├── Generate each document after confirmation
├── API documentation extracted from actual code
├── Architecture diagrams in Mermaid format
└── CHANGELOG generated from git log
```

Recipe 9: Test Suite Completion

```
You: The project currently has only 30% test coverage.
    Please help me raise the core modules to 80% coverage:

    Priority:
    1. src/server/routers/ — API routes (most important)
    2. src/services/ — Business logic
    3. src/utils/ — Utility functions
    4. src/components/ — Critical interactive components

    Requirements:
    - For each module, first analyze existing tests and find gaps
    - Tests should cover: happy paths, edge cases, error handling
    - Mock external dependencies, do not mock internal modules
    - Run tests after completing each module

Recommended: Use Agents in parallel to write tests for different module
```

Recipe 10: Dependency Upgrade

```
You: Project dependencies need major version upgrades:
    - React 18 → 19
    - Next.js 14 → 15
    - TypeScript 5.3 → 5.5

    Please:
    1. Test the upgrade in an isolated environment (worktree) first
    2. List all breaking changes and affected code
    3. Fix compatibility issues one by one
    4. Run the full test suite
    5. If everything passes, tell me how to merge

Expected behavior:
├── Create git worktree (isolated environment)
├── Upgrade dependencies in the worktree
├── npm install → check for peer dependency conflicts
├── npm run build → collect compilation errors
├── Fix each issue
├── npm test → confirm all pass
└── Return the worktree path for the user to merge
```

Part 22: Troubleshooting & Common Pitfalls

22.1 Top 10 Common Issues

#	Problem	Symptoms	Root Cause	Solution

1	**Slow responses**	Wait time goes from seconds to minutes	Context approaching the limit	`/compact` or start a new session
2	**Forgetting context**	Claude forgets earlier discussion	Auto-Compact compressed early messages	Write key information into CLAUDE.md
3	**Frequent permission prompts**	Every operation requires confirmation	Not enough allow rules	Expand rules in settings.json
4	**File edit failure**	"old_string not found"	String is not unique or has hidden characters	Provide more context to make old_string unique
5	**MCP connection failure**	MCP server not responding	Wrong command path or missing dependencies	`/mcp` to check status, verify command
6	**Sub-agent ineffective**	Agent returns empty or irrelevant results	Prompt too vague	Give specific paths, goals, and constraints
7	**Token limit exceeded**	"Context window exceeded"	Too much content in a single conversation	Split tasks, use multiple sessions
8	**Inconsistent code style**	Generated code differs from project style	CLAUDE.md lacks style guide	Add coding conventions to CLAUDE.md
9	**Tests not passing**	Tests fail after modifications	Claude does not know the project's test patterns	Describe the test framework and patterns in CLAUDE.md
10	**API costs too high**	Single session costs several dollars	Redundant searches, large file reads	Precise instructions + `/compact` + model switching

22.2 Permission-Related Troubleshooting

```
Problem: git status requires confirmation every time
┌──────────────────────────────────────┐
│ Diagnose: Check settings.json        │
│ $ cat ~/.claude/settings.json        │
│                                      │
│ Fix: Add allow rules                 │
│ "Bash(prefix:git status)"            │
│ "Bash(prefix:git diff)"              │
│ "Bash(prefix:git log)"               │
└──────────────────────────────────────┘

Problem: Cannot edit .env file (blocked by hook)
┌──────────────────────────────────────┐
│ Diagnose: Check hook configuration   │
│ /hooks                               │
│                                      │
│ Fix: Adjust the hook's matcher       │
│ or temporarily disable that hook     │
```

```
└──────────────────────────────────────┘

Problem: Commands rejected in sandbox mode
┌──────────────────────────────────────┐
│ Diagnose: SandboxManager config      │
│ Check the sandbox file access        │
│ whitelist                            │
│                                      │
│ Fix: Add required paths to the       │
│ whitelist, or disable sandbox in a   │
│ trusted environment                  │
└──────────────────────────────────────┘
```

22.3 Performance Troubleshooting

```
Problem: Claude Code startup is slow (>5 seconds)
Possible causes:
├── MCP server startup timeout → Check MCP config, disable unneeded ser
├── Large CLAUDE.md content → Trim @include files
├── Network issues → Check API connection
└── GrowthBook feature flag fetch timeout → Usually auto-times out, no

Problem: Tool execution timeout
Possible causes:
├── Bash command takes too long → Set a reasonable timeout
├── Glob too slow on large projects → Use a more precise pattern
├── MCP tool responds slowly → Check MCP server status
└── Network request timeout → WebFetch defaults to 120s, may need incre

Problem: High memory usage
Possible causes:
├── Messages accumulating in long sessions → /compact or /clear
├── Too many sub-agents running in parallel → Reduce parallelism
└── Large file reads → Use offset/limit parameters
```

22.4 CLAUDE.md Pitfalls

```
Pitfall 1: Too much content
├── Problem: CLAUDE.md + @include exceeds 40,000 characters
├── Symptom: Some content is silently truncated
└── Fix: Trim content, keep only essential instructions

Pitfall 2: Contradictory instructions
├── Problem: Global CLAUDE.md and project CLAUDE.md have conflicting in
├── Symptom: Unpredictable behavior
└── Fix: Project-level overrides global-level; ensure consistency

Pitfall 3: Over-constraining
├── Problem: Instructions like "do not modify any files"
├── Symptom: Claude cannot complete tasks
```

```
└── Fix: Constraints should be specific, e.g. "do not modify src/legacy

Pitfall 4: Circular @include references
├── Problem: A includes B, B includes A
├── Symptom: Source code has circular reference detection, which breaks
└── Fix: Check for and eliminate circular references

Pitfall 5: Including large files
├── Problem: @include a 10,000-line file
├── Symptom: Crowds out space for other content, wastes tokens
└── Fix: Reference concise document summaries only, not full files
```

22.5 Hook Debugging Tips

```
# Test whether hooks are correctly configured
/hooks

# Debug hook execution
# Add logging to the hook script
echo '{"continue":true}'
# Hook output to stderr does not affect JSON parsing
echo "DEBUG: input was $TOOL_INPUT_FILE_PATH" >&2

# Common hook errors
1. Invalid JSON output format → Hook return must be valid JSON
2. Script lacks execute permission → chmod +x hook-script.sh
3. Wrong path → Use absolute paths
4. Timeout → Default is 60s; complex operations may need a longer timeo
5. Missing environment variables → Hooks run in a separate process
```

22.6 Best Practices Checklist

```
☐ Project root has a CLAUDE.md
☐ CLAUDE.md includes tech stack, commands, directory structure, convent
☐ settings.json has reasonable allow/deny rules configured
☐ Common commands (test, lint, build) are in the allow list
☐ Dangerous commands (rm -rf /, sudo, force push) are in the deny list
☐ .claude/rules/ rule files are set up for the project type
☐ MCP servers are configured as needed (do not configure unused ones)
☐ Hooks have been tested and do not interfere with normal workflow
☐ Long tasks have a /compact strategy
☐ Team members share consistent CLAUDE.md and .claude/ configuration
```

Part 19: Key Differences from the Standard Claude API

Feature	Standard Claude API	Claude Code
Execution	API calls	CLI + interactive TUI

File access	None	Full file system
Code execution	None	Bash/PowerShell
Session state	Stateless	Persistent session storage
Tool permissions	Fixed API key	Interactive prompts + multi-mode
Context management	Fixed window	Auto-compaction + caching
Agent system	Not built-in	Full sub-agent/coordinator
MCP support	Not built-in	Full MCP client + registry
Plugins/Skills	Not supported	Full plugin system
Git integration	Not supported	Deep Git state + hooks
CLAUDE.md	Not supported	Multi-level support
Security model	None	Rules + classifier + sandbox
Cost control	Per-call	Token budget + auto-compaction
Multi-model	Manual selection	Dynamic switching + fallback

Part 20: Future Roadmap & Outlook

20.1 Known Development Directions

Future directions inferred from feature gates and code comments in the source:

```
1. KAIROS Fully Autonomous Mode
   ├── Always-on background AI assistant
   ├── Proactive task execution
   └── Voice interaction support

2. Coordinator Multi-Agent Orchestration
   ├── Multiple specialized agents collaborating
   ├── Automatic task assignment
   └── Result aggregation and conflict resolution

3. Stronger MCP Ecosystem
   ├── Official MCP registry
   ├── Full OAuth support
   └── More transport protocols

4. Context Management Evolution
   ├── Snip compaction (smarter context pruning)
   ├── Context Collapse
   └── Support for larger context windows

5. Enterprise Features
   ├── MDM policy management
   ├── Remote settings push
   ├── Compliance audit logging
   └── SSO integration
```

20.2 Model Iteration Cadence

Model launch annotations `@[MODEL LAUNCH]` in the source code show:

- `FRONTIER_MODEL_NAME = 'Claude Opus 4.6'` — Current frontier model
- Each model update requires modifying model names in multiple prompt locations
- Supports Bedrock and Vertex AI multi-platform deployment

Appendix

Appendix A: Complete Tool Parameter Reference

FileRead

```
{
  "file_path": "Absolute path (required)",
  "offset": "Starting line number (optional)",
  "limit": "Number of lines to read (optional, default 2000)",
  "pages": "PDF page range (optional, e.g. '1-5')"
}
```

FileEdit

```
{
  "file_path": "Absolute path (required)",
  "old_string": "Text to replace (required, must be unique)",
  "new_string": "Replacement text (required)",
  "replace_all": "Replace all matches (optional, default false)"
}
```

Bash

```
{
  "command": "Shell command (required)",
  "timeout": "Timeout in milliseconds (optional, default 120000, max 60
  "run_in_background": "Run in background (optional, default false)"
}
```

Agent

```
{
  "prompt": "Task description (required)",
  "description": "3-5 word short description (required)",
  "subagent_type": "Agent type (optional)",
  "model": "Model override (optional, sonnet|opus|haiku)",
  "isolation": "Isolation mode (optional, 'worktree')",
  "run_in_background": "Run in background (optional)"
```

```
}
```

Grep

```
{
  "pattern": "Regular expression (required)",
  "path": "Search directory (optional, default cwd)",
  "glob": "File filter (optional, e.g. '*.ts')",
  "type": "File type (optional, e.g. 'js', 'py')",
  "output_mode": "Output mode (optional, content|files_with_matches|cou
  "context": "Context lines (optional)",
  "head_limit": "Result limit (optional, default 250)",
  "multiline": "Multiline matching (optional, default false)"
}
```

Glob

```
{
  "pattern": "Glob pattern (required, e.g. '**/*.tsx')",
  "path": "Search directory (optional, default cwd)"
}
```

Appendix B: Complete settings.json Template

```
{
  "theme": "dark",
  "verbose": false,
  "autoCompactEnabled": true,

  "permissions": {
    "defaultMode": "default",
    "rules": {
      "allow": [
        "FileRead",
        "FileEdit",
        "FileWrite",
        "Glob",
        "Grep",
        "Bash(prefix:npm)",
        "Bash(prefix:git status)",
        "Bash(prefix:git diff)",
        "Bash(prefix:git log)",
        "Bash(prefix:ls)",
        "Bash(prefix:pwd)"
      ],
      "deny": [
        "Bash(prefix:rm -rf /)",
        "Bash(prefix:sudo)",
        "Bash(prefix:git push --force)"
```

```
      ],
      "ask": []
    }
  },

  "env": {
    "NODE_ENV": "development"
  },

  "mcpServers": {},

  "hooks": [],

  "preferredNotifChannel": "system"
}
```

Appendix C: CLAUDE.md Best Practices Template

```
# Project: [Project Name]

## Overview
[One-sentence project description]

## Tech Stack
- Language: [TypeScript/Python/...]
- Framework: [Next.js/FastAPI/...]
- Database: [PostgreSQL/MongoDB/...]
- Package Manager: [npm/pnpm/poetry/...]

## Directory Structure
- src/ — Source code
- tests/ — Tests
- docs/ — Documentation
- scripts/ — Scripts

## Development Commands
- Start: `npm run dev`
- Test: `npm test`
- Build: `npm run build`
- Lint: `npm run lint`

## Coding Standards
- [List key standards]

## Important Notes
- [List items requiring special attention]

## Referenced Documentation
@./docs/architecture.md
@./docs/api.md
```

Appendix D: Key Source File Index

Function	File Path	Lines
CLI entry	`src/entrypoints/cli.tsx`	~300
Main orchestrator	`src/main.tsx`	~4,683
Query engine	`src/QueryEngine.ts`	~200+
Core loop	**`src/query.ts`**	**~1,729**
Tool definition	`src/Tool.ts`	~200+
Tool registration	`src/tools.ts`	~300+
Streaming executor	`src/services/tools/StreamingToolExecutor.ts`	~200+
Permission engine	`src/utils/permissions/permissions.ts`	~250+
YOLO classifier	`src/utils/permissions/yoloClassifier.ts`	52KB
Hook execution	`src/utils/hooks/execAgentHook.ts`	~200+
Hook types	`src/types/hooks.ts`	~300+
MCP client	`src/services/mcp/client.ts`	119KB
MCP types	`src/services/mcp/types.ts`	~200+
API client	`src/services/api/claude.ts`	125KB
Retry logic	`src/services/api/withRetry.ts`	28KB
Context compaction	`src/services/compact/compact.ts`	~100+
Auto-compaction	`src/services/compact/autoCompact.ts`	~100+
Config loading	`src/utils/config.ts`	~250+
CLAUDE.md	`src/utils/claudemd.ts`	~200+
Prompt construction	`src/constants/prompts.ts`	~300+
Prompt caching	`src/constants/systemPromptSections.ts`	~60
System prompt	`src/utils/systemPrompt.ts`	~124
Message types	`src/types/message.ts`	~100+
Permission types	`src/types/permissions.ts`	~250+
App state	`src/state/AppStateStore.ts`	~150+
Command registration	`src/commands.ts`	~200+
Token counting	`src/utils/tokens.ts`	~100+
Context analysis	`src/utils/analyzeContext.ts`	42KB
Build script	`scripts/build.mjs`	~230
Agent tool	`src/tools/AgentTool/AgentTool.tsx`	233KB
Agent loading	`src/tools/AgentTool/loadAgentsDir.ts`	26KB

Appendix E: Frequently Asked Questions

Q1: How large is Claude Code's context window?
A: It depends on the model in use. Claude Opus 4.6 supports a 1M token context. The system automatically manages context and triggers compaction as it approaches the limit.

Q2: How can I reduce API costs?
A:

1. Use CLAUDE.md to reduce exploratory token consumption
2. Use specific file paths instead of searching
3. Use /compact promptly to compress context
4. Use the Haiku model for simple tasks

Q3: What is the difference between hooks and MCP?
A: Hooks inject logic at internal Claude Code event points (e.g., before/after tool calls), while MCP extends available external tools and data sources through a standard protocol.

Q4: How many levels deep can sub-agents be nested?
A: There is no hard limit in the source code, but each level of sub-agent consumes token budget. In practice, it is recommended not to exceed 2-3 levels.

Q5: How much content can CLAUDE.md hold?
A: The source code defines `MAX_MEMORY_CHARACTER_COUNT = 40,000` characters. Content beyond this limit is truncated.

Acknowledgments

This whitepaper is based on source code analysis from the following public repositories:

- sanbuphy/claude-code-source-code
- ChinaSiro/claude-code-sourcemap

All rights reserved by the author. For technical learning and research purposes only.